DATA PROFILER

Part A: Fundamentals Theory Document

---

1. Write a Short Note: What is Data Analysis?

Data Analysis is a systematic process of inspecting, cleaning, transforming, and modeling raw data with the objective of discovering actionable insights, drawing meaningful conclusions, and supporting strategic decision-making. In today's data-driven world, organizations collect massive volumes of unstructured and structured data. Without proper analysis, this data remains raw and unutilized. Data Analysis bridges the gap between raw data and business intelligence.

Key Types of Data Analysis:

- **Descriptive Analysis:** Answers 'What happened?'. It uses historical data, summary statistics, and visualization (like charts and histograms) to describe the current or past state of the data. Examples include calculating average customer spend, monthly sales, or churn rates.
- **Diagnostic Analysis:** Answers 'Why did it happen?'. It deep-dives into historical data to identify patterns, anomalies, and root causes. For example, investigating if a drop in customer retention was caused by a recent application crash, price increase, or competitors' campaigns.
- **Predictive Analysis:** Answers 'What is likely to happen?'. It uses statistical models, historical patterns, and Machine Learning algorithms to forecast future trends. For example, predicting customer lifetime value, or identifying which customers are at high risk of churning next month.
- **Prescriptive Analysis:** Answers 'What should we do about it?'. It combines insights from all previous analyses to recommend specific courses of action or automate decision-making. For example, auto-triggering discount coupons to customers predicted as highly likely to churn.

2. How to Plan a Data Science Project: Step-by-Step

A successful Data Science project is executed through a structured lifecycle. Following a disciplined pipeline ensures that the project delivers tangible business value and avoids technical debt. Below are the key steps involved:

Step 1: Business Understanding & Goal Definition

Define the objective clearly in plain language. Understand the problem you are solving, the stakeholders involved, and the metrics for success (e.g., reduce customer churn rate by 10% in the next quarter).

Step 2: Data Acquisition & Collection

Gather data from multiple relevant sources. This involves writing SQL queries to retrieve transaction databases, consuming REST APIs for real-time customer feedback, loading CSV/JSON exports, or scraping web portals.

Step 3: Data Cleaning & Preprocessing

Real-world data is dirty. This critical step handles missing data points (imputation or deletion), removes duplicates, handles extreme outliers, and corrects incorrect data types (e.g., converting a date column from string to datetime object).

Step 4: Exploratory Data Analysis (EDA)

Analyze the datasets visually and statistically to uncover initial patterns, trends, and correlations. Create visualizations like histograms, correlation heatmaps, box plots, and Q-Q plots to understand distributions and variables.

Step 5: Feature Engineering & Selection

Transform raw variables into meaningful features that help ML models learn better. This includes standardizing/scaling numerical columns, encoding categorical variables (One-Hot Encoding, Label Encoding), and creating new interaction features (e.g., dividing total spending by purchase frequency to get 'average order value').

Step 6: Model Training & Selection

Select appropriate machine learning algorithms depending on the task type (e.g., Classification, Regression, or Clustering). Split the dataset into training (e.g., 80%) and testing (e.g., 20%) sets. Train candidate models (like Logistic Regression, Random Forests, XGBoost).

Step 7: Model Evaluation

Evaluate model performance on unseen test data using domain-specific metrics. For classification tasks, use Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Select the champion model that balances bias and variance.

Step 8: Hyperparameter Tuning

Optimize the performance of the selected champion model by fine-tuning its internal parameters. Use techniques like Grid Search Cross-Validation (GridSearchCV) or Randomized Search to search the parameter space systematically.

Step 9: Model Deployment & Monitoring

Deploy the finalized model to a production environment (as an API endpoint, batch job, or

integrated dashboard component). Set up monitoring pipelines to track model drift, performance degradation, and data shift over time.

3. Framing a Machine Learning Problem Statement: Customer Churn Prediction

Problem Statement	A consumer insights company wants to reduce customer attrition by identifying which customers are at high risk of stopping their purchases (churning) in the near future. This enables the marketing team to target them with proactive retention campaigns (e.g., custom discount offers, feedback surveys).
Input Data (Features / Independent Variables)	- Transactional Data: Purchase frequency, average transaction value, recency (days since last purchase), total order cancellations. - Customer Demographics: Age, gender, location, account tenure (months active). - Platform Engagement: App login frequency, active session duration, click-through rates. - Support Logs: Number of open/closed customer service tickets, sentiment score from support chat transcripts (via API).
Output Data (Target Variable / Label)	Churn: A binary indicator column. - 1: Customer churned (did not make a purchase in the defined churn period, e.g., past 30 days). - 0: Customer active (made at least one purchase in the last 30 days).
Machine Learning Problem Type	Supervised Binary Classification. (The goal is to map input features to a discrete binary class label: 0 or 1).
Key Evaluation Metrics	- Recall (Sensitivity): Critical to maximize, because missing a customer who is going to churn (False Negative) is very costly for the business. - F1-Score: Used to balance Precision (avoiding false alarms on loyal customers) and Recall. - ROC-AUC: To evaluate the model's ability to rank risk probabilities across various decision thresholds.
ML Problem Structure (Python Representation)	A structured dictionary mapping the problem's components:

```
# Machine Learning Problem Structure Definition
customer_churn_problem = {
    "problem_name": "Customer Churn Prediction",
```

```
    "ml_type": "Binary Classification",
    "features": [
        "recency_days", "purchase_frequency", "avg_spend_amount",
        "cancellation_rate", "tenure_months", "customer_age",
        "support_tickets_count", "chat_sentiment_score"
    ],
    "target": "is_churned",  # Binary (0: Active, 1: Churned)
    "primary_metric": "Recall",
    "secondary_metrics": ["F1-Score", "ROC-AUC"]
}
```

4. Technical Explanation: What are Tensors?

A Tensor is a mathematical object that generalizes scalars, vectors, and matrices to higher dimensions. In computer science and machine learning, a tensor is fundamentally a multi-dimensional array of numbers. Tensors act as the core data structure in deep learning frameworks like TensorFlow and PyTorch. The dimensionality of a tensor is referred to as its Rank (or Order/Axes).

Tensor Dimensions & Ranks:

- **0D Tensor (Scalar):** Rank 0. A single, isolated number. It has no axes. Example: Temperature (e.g., 25), Age (e.g., 34). Shape: `()`
- **1D Tensor (Vector):** Rank 1. An array or list of numbers representing a single axis. Example: A customer's feature row `[34, 1, 150.5]` (representing Age, Gender, Spend). Shape: `(n,)`
- **2D Tensor (Matrix):** Rank 2. A table of numbers with rows and columns (2 axes). Example: A tabular dataset where each row is a customer and each column is a feature. Shape: `(rows, columns)`
- **3D Tensor:** Rank 3. A cube of numbers representing three axes. Example: A time-series batch of data (samples, timesteps, features), or a grayscale image (width, height, 1 channel). Shape: `(axis1, axis2, axis3)`
- **4D Tensor:** Rank 4. A collection of 3D tensors. Example: A batch of color images. Shape: `(batch_size, height, width, channels)` where channels represent Red, Green, and Blue color values.

In-Depth Tensor Implementation Examples using NumPy

NumPy provides the `ndarray` object, which is Python's standard implementation of a tensor. Below is Python code demonstrating how to construct and inspect tensors of different dimensions using NumPy:

```
import numpy as np
```

```
# 1. Scalar (0D Tensor)
scalar = np.array(42)
print(f"Scalar Value: {scalar}")
print(f"Rank/NDim: {scalar.ndim}, Shape: {scalar.shape}\n")

# 2. Vector (1D Tensor)
vector = np.array([12, 24, 36, 48])
print(f"Vector: {vector}")
print(f"Rank/NDim: {vector.ndim}, Shape: {vector.shape}\n")

# 3. Matrix (2D Tensor)
matrix = np.array([
    [5.1, 3.5, 1.4],
    [4.9, 3.0, 1.4],
    [6.2, 3.4, 5.4]
])
print("Matrix:\n", matrix)
print(f"Rank/NDim: {matrix.ndim}, Shape: {matrix.shape}\n")

# 4. 3D Tensor (e.g. Time-Series or Small Grayscale Image)
tensor_3d = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]],
    [[9, 10], [11, 12]]
])
print("3D Tensor:\n", tensor_3d)
print(f"Rank/NDim: {tensor_3d.ndim}, Shape: {tensor_3d.shape}\n")

# 5. 4D Tensor (e.g. Batch of 2 Color Images, each 2x2 pixels with 3 RGB
channels)
tensor_4d = np.zeros((2, 2, 2, 3))
# Fill with sample values
tensor_4d[0, :, :, 0] = 255  # Red channel for first image
tensor_4d[1, :, :, 1] = 255  # Green channel for second image
print(f"4D Tensor Rank/NDim: {tensor_4d.ndim}, Shape: {tensor_4d.shape}")
```